

Linear Regression for Classification

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Give categories their own model output, via one-hot labels and softmax
- Define cross-entropy loss for that output
- Show its gradient looks just like linear regression's

Classification

The Big Question

Predicting *which*, not *how much*

What we have vs. what's different

What we already have

- A dataset, a model, a loss
- A way to train it — all from last lecture
- All of it assumed the answer is a single number

What's different now

- The answer is a **category**
 - spam or not spam
 - cat or dog
 - which digit
- No natural “how far off” between categories, the way there is between numbers

The Big Idea

Some questions don't have a numeric answer — they have a category as the answer.

A concrete example we'll carry through

We'll sort pieces of fruit into **apple**, **banana**, or **orange**, from two features:

weight (g)	color score (0=green, 1=red)	fruit
150	0.9	apple
120	0.2	banana
180	0.7	orange

Why not just reuse linear regression directly?

Categories aren't numbers

- We're used to fitting a line to numbers
- Apple, banana, orange aren't numbers — there's no natural “less than” or “far apart” between them
- Coding them as 0, 1, 2 invents structure that isn't really there

The Big Idea

Each *individual* category is binary — it is, or it isn't.

We just need to find the most likely

Let's just try it anyway

- Code the labels as integers: apple = 0, banana = 1, orange = 2
- Fit the usual linear model against those codes:

$$\hat{y} = \mathbf{w} \cdot \mathbf{x} + b$$

- Predict a class by rounding $\hat{y}$ to the nearest code

Problem 1: we invented an ordering

- This encoding says banana (1) sits *between* apple (0) and orange (2)
- It says orange *minus* apple equals two *whole* bananas
- Relabel the fruits in a different order and the model's behavior changes

Problem 2: the output doesn't match the question

- $\hat{y}$ can be anything — -1.3 , 4.7 , whatever the line gives
- Rounding to the nearest code is a patch, not a fix
- A 1.99 and a 1.01 both round to “banana,” but they don't feel equally confident

What we actually need

- A way to write “which category” without implying order or distance → **one-hot encoding**
- A model output that’s bounded and behaves like a confidence → **softmax**

One-hot encoding

The Big Idea

Give every category its own slot, and just mark which one is true.

Our fruit, one-hot

fruit	apple	banana	orange
apple	1	0	0
banana	0	1	0
orange	0	0	1

Note

Compare to the integer coding from a few slides ago — apple, banana, and orange no longer have any numeric relationship to each other at all

Written out

- K categories $\rightarrow$ a vector of length K , one slot per category
- Slot j is 1 if the example belongs to category j , 0 otherwise:

$$y_j = \begin{cases} 1 & \text{example is class } j \\ 0 & \text{otherwise} \end{cases}$$

- Stack the slots into one label vector $\mathbf{y} \in \{0, 1\}^K$

Exactly one 1

- Every one-hot vector has exactly one 1 — that’s the “hot” one — and the rest are 0
- The *position* of the 1 carries all the information; nothing else in the vector does
- No leftover magnitude, no leftover order — just “is it this one, or not”

A “linear” model, per class

Linear scores — for now

The Big Idea

**Score every class with a dot product — but
a score isn't a prediction yet.**

One score per class (recap from lecture 04)

- One shared model, not K separate ones — $\mathbf{W}$ has one row per class, $\mathbf{b}$ one entry per class:

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}, \quad o_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$$

- Same forward pass as multi-output regression from last lecture — the K outputs are now “ K classes” instead of “ K numbers”

These scores are called logits

- o_j is a **logit** — the model's raw, unnormalized opinion about class j
- Bigger o_j means the model favors class j more, *relative to the others*
- A logit is **not** a probability: it can be negative, it has no upper bound, and the o_j 's don't sum to anything meaningful

Worked example: our fruit

Suppose, for one particular (weight, color) example, evaluating $o_j = \mathbf{w}_j \cdot \mathbf{x} + b_j$ for each class gives:

apple	banana	orange
-0.3	2.1	0.6

- Banana has the highest logit, so *right now* the model favors banana

 Warning

So the answer is just “banana,” done? That’s not actually what we do — why not?

Softmax

From scores to probabilities

The Big Idea

We make each prediction *act* like a probability – without changing which class wins.

The softmax function

$$\hat{y}_j = \text{softmax}(\mathbf{o})_j = \frac{e^{o_j}}{\sum_{k=1}^K e^{o_k}}$$

- Takes in all K logits at once, returns one probability per class
- $\hat{\mathbf{y}}$ is now our model's actual prediction — a full probability distribution over the classes

Breakdown: numerator and denominator

Numerator

$$e^{o_j}$$

- Exponentiate *this* class's logit
- Always positive
- Same ranking as o_j

Denominator

$$\sum_{k=1}^K e^{o_k}$$

- The *same kind of term*
- Summed over every class
- Includes j itself

Note

Divide one by the other: squeezes every class's share into $[0, 1]$, and all K shares add up to exactly 1

So what does this actually do?

- It's not just squashing into $[0, 1]$ — exponentiating also **magnifies** the gaps between logits
- Our banana/orange logits differed by only 1.5; their probabilities differ by a factor of $e^{1.5} \approx 4.5$
- A modest lead in logits turns into a much larger lead in confidence

Worked example: our fruit

Recall our logits: apple = -0.3 , banana = 2.1 , orange = 0.6

	apple	banana	orange
e^{o_j}	0.74	8.17	1.82
$\hat{y}_j$	0.07	0.76	0.17

- Same winner as before (banana) — softmax never changes who wins
- But now we also know *how confident*: about 76%, not just “highest”

The special case: two classes

- With only $K = 2$ classes, softmax simplifies to:

$$\hat{y}_1 = \frac{1}{1 + e^{-(o_1 - o_2)}}$$

- That's the **sigmoid** function, applied to the *difference* between the two logits
- This special case has its own name — **logistic regression** — softmax is its generalization to more than two classes

What softmax buys us

- **A real probability distribution:** non-negative, sums to 1, ranks classes the same way the logits did
- **A smooth function:** softmax is differentiable, unlike `argmax`
 - That matters a lot! To train with gradient descent, we need a loss we can actually take a gradient of

Cross Entropy

Loss function for multi-classification

The Big Idea

Placeholder: penalize the model for how little probability it gave the correct answer.

Why not just use MSE here?

The Big Idea

Placeholder: the loss should punish confident wrongness harder than MSE does.

Differentiating the loss

The Big Idea

Placeholder: this scary-looking loss has a gradient just as simple as linear regression's.

Putting it together: training a classifier

The Big Idea

Placeholder: nothing about the training loop changed — only the loss and what “prediction” means.

Wrap